

Sprint 13 Structured Notes: Keys, Relationships, and Joins

1. Current Progress

You are now moving from basic SQL into linked table design.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have completed these stages:

Sprint 1: Tools and VS Code

You learned about:

- code editors and IDEs
- VS Code
- workspace
- project folders
- Live Server
- basic shortcuts and extensions

Simple meaning: Sprint 1 prepared your coding environment.

Sprint 2: Terminal, Shell, and Command Line Basics

You learned about:

- terminal
- command line
- shell
- Bash
- command history
- auto-complete
- clearing and interrupting commands

Simple meaning: Sprint 2 taught you what happens when you type commands.

Sprint 3: Bash File and Folder Commands

You learned how to:

- move between folders
- create files and folders
- rename, copy, and delete files
- read files
- use flags and options

Simple meaning: Sprint 3 taught you how to control files and folders from the terminal.

Sprint 4: Bash Scripting Basics

You learned about:

- Bash scripts
- .sh files
- shebang
- interpreter paths
- variables
- arrays
- loops
- do and done
- variable interpolation

Simple meaning: Sprint 4 taught you how to save terminal commands into reusable script files.

Sprint 5 to Sprint 10: Git Foundation

You learned:

- Git setup
- local Git workflow
- GitHub repositories and remotes
- branches and pull requests
- forks and sync
- .gitignore
- merge conflicts
- safer Git recovery tools
- beginner CI/CD ideas

Simple meaning: Sprints 5 to 10 taught you how developers track, share, review, and protect project changes.

Sprint 11: Relational Databases and PostgreSQL Setup

You learned:

- what a relational database is
- table, row, and column
- record, attribute, and field
- schema
- primary key and foreign key basics
- relational vs non-relational databases
- PostgreSQL
- pgAdmin
- psql
- PostgreSQL setup basics
- how to connect to the default postgres database

Simple meaning: Sprint 11 gave you the database foundation and PostgreSQL setup.

Sprint 12: SQL Basics

You learned:

- what SQL is
- SQL vs PostgreSQL
- statements and queries
- semicolons
- psql shortcut commands
- CREATE DATABASE
- CREATE TABLE
- snake_case naming
- data types
- INSERT INTO
- VALUES
- SELECT
- WHERE
- comparison operators such as `=` and `<`

Simple meaning: Sprint 12 taught you how to create simple tables, insert rows, and query basic data.

Sprint 13: Keys, Relationships, and Joins

Now you are learning how tables connect to each other.

In Sprint 12, you mainly worked with one table at a time. In Sprint 13, you now learn how to:

- understand keys more deeply
- connect tables using foreign keys
- understand relationship types

- create many-to-many structures using a junction table
- combine data from multiple tables using JOIN

Simple meaning: Sprint 13 teaches you how to design linked tables and read connected data.

2. What Sprint 13 Is About

Sprint 13 is about the next level of SQL structure.

Now you start learning:

- what a primary key really does
- what a composite primary key is
- what a foreign key is
- how foreign keys prevent broken links
- what orphaned records are
- one-to-one relationships
- one-to-many relationships
- many-to-one relationships
- many-to-many relationships
- self-referencing relationships
- junction tables
- JOIN operations
- INNER JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN

Simple meaning: Sprint 13 teaches you how data in one table can be connected to data in another table.

3. Main Goal of Sprint 13

The main goal of Sprint 13 is:

Design linked tables and combine their data.

By the end of Sprint 13, you should be able to:

- explain what a primary key is
- explain what a composite primary key is
- explain what a foreign key is
- understand that a table can have multiple foreign keys

- explain what orphaned records are
- explain one-to-one, one-to-many, many-to-one, many-to-many, and self-referencing relationships
- explain why many-to-many needs a junction table
- understand that JOIN combines related data from tables
- use INNER JOIN
- use LEFT JOIN / LEFT OUTER JOIN
- use RIGHT JOIN / RIGHT OUTER JOIN
- use FULL OUTER JOIN
- understand SELF JOIN
- understand CROSS JOIN

Simple meaning: Sprint 13 teaches you how to connect tables properly and read connected information.

4. Why Sprint 13 Is Important

Sprint 13 is important because real databases usually do not keep everything in one table.

Real systems often have tables like:

- customers
- orders
- products
- categories
- students
- courses
- authors
- books

These tables are linked.

Example:

- one customer can have many orders
- one order belongs to one customer
- one student can join many courses
- one course can have many students

Without keys and relationships, the data becomes messy and unreliable.

Without JOIN, you cannot easily read connected information.

Simple meaning: Sprint 13 is important because real projects use connected tables, not isolated tables.

5. Big Beginner Idea

Think like this:

- **table** = one data group
- **primary key** = unique ID of a row
- **foreign key** = link to another table
- **relationship** = how tables connect
- **JOIN** = way to read connected data together

Simple example:

```
customers table
customer_id | name
1           | Amal
2           | Nimal

orders table
order_id | customer_id | item
101      | 1           | Laptop
102      | 2           | Mouse
```

Here:

- `customer_id` is the main ID in `customers`
- `customer_id` inside `orders` links to `customers`

So the database can understand:

- order 101 belongs to Amal
- order 102 belongs to Nimal

Simple meaning: Keys tell the database who each row is and how rows from different tables are connected.

6. Why Keys Are Important

Imagine a school system.

You have students:

```
student_id | name
1           | Amal
2           | Nimal
```

And course enrollments:

student_id	course_id
1	10
2	20

How does the database know which enrollment belongs to which student?

It uses the `student_id`.

That is why keys are important.

Simple meaning: Keys are IDs that help the database identify rows and connect tables.

7. Primary Key

A **primary key** is a column that uniquely identifies each row in a table.

Example:

```
CREATE TABLE students (  
  student_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL  
);
```

Here:

- `student_id` is the unique ID column
- `SERIAL` gives auto-numbering
- `PRIMARY KEY` marks it as the main ID

Example data:

student_id	name
1	Amal
2	Nimal
3	Kamal

A primary key:

- must be unique
- cannot be duplicated

- cannot be `NULL`
- identifies one exact row

Simple meaning: A primary key is the unique ID of a row.

8. Composite Primary Key

A normal primary key can use one column.

Example:

```
student_id SERIAL PRIMARY KEY
```

But a **composite primary key** uses two or more columns together.

Example:

```
CREATE TABLE course_enrollments (  
  student_id INTEGER NOT NULL,  
  course_id INTEGER NOT NULL,  
  PRIMARY KEY (student_id, course_id)  
);
```

This means the combination of `student_id` and `course_id` must be unique.

Allowed:

student_id	course_id
1	10
1	20
2	10

Not allowed twice:

student_id	course_id
1	10
1	10

Simple meaning: A composite primary key uses multiple columns together to make one unique identity.

9. Foreign Key

A **foreign key** is a column that references the primary key of another table.

Example:

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE orders (  
  order_id SERIAL PRIMARY KEY,  
  customer_id INTEGER NOT NULL,  
  item VARCHAR(100) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

This means:

- `customers.customer_id` is the main ID in the customers table
- `orders.customer_id` must match an existing customer

Example data:

```
customers table  
customer_id | name  
1           | Amal  
2           | Nimal
```

```
orders table  
order_id | customer_id | item  
101      | 1           | Laptop  
102      | 2           | Mouse
```

So:

- order 101 belongs to Amal
- order 102 belongs to Nimal

Simple meaning: A foreign key connects one table to another table.

10. A Table Can Have Multiple Foreign Keys

A table is not limited to one foreign key.

Example idea:

```
order_items table
order_id
product_id
quantity
```

Here:

- `order_id` can be a foreign key to `orders`
- `product_id` can be a foreign key to `products`

So one table can connect to multiple other tables.

Simple meaning: A table can have many links to other tables.

11. Orphaned Records

An **orphaned record** is a row that points to something that does not exist.

Example:

```
orders table
order_id | customer_id | item
103      | 99           | Keyboard
```

But there is no customer with ID 99.

This is a broken link.

A foreign key helps prevent this kind of problem.

Simple meaning: An orphaned record is a child row that points to a missing parent row.

12. Relationship Types

12.1 One-to-One Relationship

One row in table A connects to one row in table B.

Examples:

- one person → one passport
- one user → one profile

Simple meaning: One item connects to only one other item.

12.2 One-to-Many Relationship

One row in table A connects to many rows in table B.

Examples:

- one customer → many orders
- one teacher → many classes
- one category → many products

Example:

```
customers
customer_id | name
1           | Amal

orders
order_id | customer_id
101      | 1
102      | 1
103      | 1
```

Amal has many orders.

Simple meaning: One parent row can have many child rows.

12.3 Many-to-One Relationship

This is the opposite view of one-to-many.

Examples:

- many orders → one customer
- many products → one category

Simple meaning: Many rows can belong to one parent row.

12.4 Many-to-Many Relationship

Many rows in table A can connect to many rows in table B.

Examples:

- many students → many courses
- many books → many authors
- many products → many suppliers

A student can join many courses. A course can have many students.

This cannot be stored cleanly using only two tables.

That is why we need a third table called a **junction table**.

Simple meaning: Many-to-many needs a middle table to store the connections.

12.5 Self-Referencing / Recursive Relationship

A self-referencing relationship means a table connects to itself.

Example:

- one employee can manage another employee

Example table:

```
CREATE TABLE employees (  
    employee_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    manager_id INTEGER,  
    FOREIGN KEY (manager_id) REFERENCES employees(employee_id)  
);
```

Example data:

employee_id	name	manager_id
1	Amal	NULL
2	Nimal	1
3	Kamal	1

This means:

- Amal is the manager
- Nimal reports to Amal
- Kamal reports to Amal

Simple meaning: A self-referencing relationship means rows in the same table connect to other rows in that same table.

13. Junction Table

A **junction table** is a middle table used in a many-to-many relationship.

Example:

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);

CREATE TABLE course_enrollments (
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Here:

- `students` stores students
- `courses` stores courses
- `course_enrollments` stores which student joins which course

Simple meaning: A junction table stores connections between two many-to-many tables.

14. What Is a JOIN?

A **JOIN** combines related information from two tables.

Example:

```
customers table
customer_id | name
1           | Amal
2           | Nimal
```

```
orders table
order_id | customer_id | item
101      | 1           | Laptop
102      | 2           | Mouse
```

A JOIN can produce something like this:

```
order_id | customer_name | item
101      | Amal          | Laptop
102      | Nimal         | Mouse
```

Simple meaning: A JOIN lets SQL read connected data from different tables together.

15. JOIN Condition with ON

When joining tables, SQL needs to know how the tables match.

That is what the **ON** condition does.

Example:

```
SELECT *
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

Here:

```
ON customers.customer_id = orders.customer_id
```

means:

- match rows where the customer IDs are the same

Simple meaning: `ON` tells SQL which columns connect the two tables.

16. INNER JOIN

`INNER JOIN` keeps only matching rows from both tables.

Example:

```
SELECT *  
FROM products  
INNER JOIN sales  
ON products.product_id = sales.product_id;
```

If a product has no matching sale, it will not appear. If a sale has no matching product, it will not appear.

Simple meaning: `INNER JOIN` shows only rows that match in both tables.

17. LEFT JOIN / LEFT OUTER JOIN

`LEFT JOIN` returns all rows from the left table and matching rows from the right table.

Example:

```
SELECT *  
FROM products  
LEFT JOIN sales  
ON products.product_id = sales.product_id;
```

What happens:

- every product appears
- matching sales appear when available

- if there is no sale, the sales columns become `NULL`

Simple meaning: `LEFT JOIN` shows everything from the left table, even if there is no match on the right.

Note: `LEFT JOIN` and `LEFT OUTER JOIN` mean the same thing here.

18. RIGHT JOIN / RIGHT OUTER JOIN

`RIGHT JOIN` returns all rows from the right table and matching rows from the left table.

Example:

```
SELECT *  
FROM products  
RIGHT JOIN sales  
ON products.product_id = sales.product_id;
```

What happens:

- every sale appears
- matching product details appear when available
- if there is no product match, product columns become `NULL`

Simple meaning: `RIGHT JOIN` shows everything from the right table, even if there is no match on the left.

Note: `RIGHT JOIN` and `RIGHT OUTER JOIN` mean the same thing here.

19. FULL OUTER JOIN

`FULL OUTER JOIN` returns all rows from both tables.

Example:

```
SELECT *  
FROM products  
FULL OUTER JOIN sales  
ON products.product_id = sales.product_id;
```

What happens:

- matching rows are joined

- left-only rows still appear
- right-only rows still appear
- missing parts become `NULL`

Simple meaning: `FULL OUTER JOIN` shows everything from both tables, even when there is no match.

20. SELF JOIN

A **SELF JOIN** joins a table with itself.

This is useful for self-referencing relationships.

Example:

```
SELECT
    employee.name AS employee_name,
    manager.name AS manager_name
FROM employees AS employee
LEFT JOIN employees AS manager
ON employee.manager_id = manager.employee_id;
```

This lets you see:

- employee name
- manager name

Example meaning:

```
employee → manager
```

Simple meaning: SELF JOIN is used when one row in a table is related to another row in the same table.

21. CROSS JOIN

A **CROSS JOIN** creates all possible row combinations between two tables.

Example:

```
SELECT *  
FROM products  
CROSS JOIN colors;
```

If you have:

```
3 products  
2 colors
```

The result becomes:

```
3 × 2 = 6 combinations
```

Important point:

- CROSS JOIN does not need a join condition like `ON`

Simple meaning: CROSS JOIN combines every row with every row.

Beginner warning: If the tables are large, the result can become very large.

22. NULL in Join Results

When joins cannot find a matching row, SQL often fills the missing side with `NULL`.

Example with `LEFT JOIN`:

customer_id	name	item
1	Amal	Laptop
2	Nimal	Mouse
3	Kamal	NULL

This means Kamal exists, but there is no matching order row.

Simple meaning: `NULL` in join results often means no matching row was found on that side.

23. Sprint 13 Practice Flow

Run this slowly in `psql`.

Step 1: Create a database

```
CREATE DATABASE sprint13_db;
```

Connect to it:

```
\c sprint13_db
```

Step 2: Create customers and orders

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INTEGER NOT NULL,  
    item VARCHAR(100) NOT NULL,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Meaning:

- `customers` is the parent table
- `orders` is the child table
- `orders.customer_id` links to `customers.customer_id`

Step 3: Insert sample data

```
INSERT INTO customers (name)  
VALUES  
( 'Amal' ),  
( 'Nimal' ),  
( 'Kamal' );  
  
INSERT INTO orders (customer_id, item)  
VALUES  
(1, 'Laptop'),
```

```
(1, 'Mouse'),  
(2, 'Keyboard');
```

Meaning:

- Amal has two orders
- Nimal has one order
- Kamal has no order yet

Step 4: Use INNER JOIN

```
SELECT customers.name, orders.item  
FROM customers  
INNER JOIN orders  
ON customers.customer_id = orders.customer_id;
```

Expected idea:

```
Amal | Laptop  
Amal | Mouse  
Nimal | Keyboard
```

Kamal does not appear because INNER JOIN shows only matches.

Step 5: Use LEFT JOIN

```
SELECT customers.name, orders.item  
FROM customers  
LEFT JOIN orders  
ON customers.customer_id = orders.customer_id;
```

Expected idea:

```
Amal | Laptop  
Amal | Mouse  
Nimal | Keyboard  
Kamal | NULL
```

Now Kamal appears too because LEFT JOIN keeps all rows from the left table.

Step 6: Create students, courses, and enrollments

```
CREATE TABLE students (  
    student_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE courses (  
    course_id SERIAL PRIMARY KEY,  
    course_name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE course_enrollments (  
    student_id INTEGER NOT NULL,  
    course_id INTEGER NOT NULL,  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id)  
);
```

Meaning:

- `students` stores student data
- `courses` stores course data
- `course_enrollments` stores each student-course connection

This is the many-to-many example.

24. Common Beginner Mistakes

Mistake 1: Thinking every table should store everything

Problem: one giant table becomes messy. Better habit: split data into focused tables and connect them with keys.

Mistake 2: Forgetting the primary key

Problem: rows do not have a clear unique identity. Better habit: every relational table should have a primary key.

Mistake 3: Using a foreign key value that does not exist

Problem: broken connection or orphaned record. Better habit: link child rows only to real parent rows.

Mistake 4: Forgetting that many-to-many needs a junction table

Problem: bad structure and repeated data. Better habit: use a middle table such as `course_enrollments`.

Mistake 5: Using the wrong join type

Problem: missing rows or unexpected output. Better habit: decide first whether you want only matches or also unmatched rows.

Mistake 6: Forgetting the `ON` condition

Problem: SQL does not know how to match the tables correctly. Better habit: always clearly define the join condition.

Mistake 7: Being confused by `NULL` after a join

Problem: you think the row is broken. Better habit: remember that `NULL` often means there was no match on one side.

Mistake 8: Using `CROSS JOIN` carelessly

Problem: the output becomes very large. Better habit: use `CROSS JOIN` only when you really want every possible combination.

25. Sprint 13 Command and Syntax Summary

Create linked tables

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE orders (  
  order_id SERIAL PRIMARY KEY,  
  customer_id INTEGER NOT NULL,  
  item VARCHAR(100) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Composite primary key

```
PRIMARY KEY (student_id, course_id)
```

INNER JOIN

```
SELECT *  
FROM customers  
INNER JOIN orders  
ON customers.customer_id = orders.customer_id;
```

LEFT JOIN

```
SELECT *  
FROM customers  
LEFT JOIN orders  
ON customers.customer_id = orders.customer_id;
```

RIGHT JOIN

```
SELECT *  
FROM customers  
RIGHT JOIN orders  
ON customers.customer_id = orders.customer_id;
```

FULL OUTER JOIN

```
SELECT *  
FROM customers  
FULL OUTER JOIN orders  
ON customers.customer_id = orders.customer_id;
```

SELF JOIN

```
SELECT *  
FROM employees AS employee  
LEFT JOIN employees AS manager  
ON employee.manager_id = manager.employee_id;
```

CROSS JOIN

```
SELECT *  
FROM products  
CROSS JOIN colors;
```

26. Easy Memory Section

Use these simple memory lines:

- **Primary key** = unique row ID
- **Composite primary key** = unique ID made from multiple columns
- **Foreign key** = link to another table's primary key
- **Orphaned record** = broken child row with no real parent
- **One-to-one** = one connects to one
- **One-to-many** = one connects to many
- **Many-to-one** = many belong to one
- **Many-to-many** = many connect to many
- **Junction table** = middle table for many-to-many
- **JOIN** = read connected data together
- **INNER JOIN** = only matches
- **LEFT JOIN** = everything from left + matches from right
- **RIGHT JOIN** = everything from right + matches from left
- **FULL OUTER JOIN** = everything from both sides
- **SELF JOIN** = join a table to itself
- **CROSS JOIN** = every row with every row

27. Sprint 13 Checkpoint

You pass Sprint 13 when you can:

- explain primary key in simple words
- explain foreign key in simple words
- explain composite primary key
- explain orphaned records
- explain the main relationship types
- explain why many-to-many needs a junction table
- explain what JOIN does
- explain the difference between INNER JOIN and LEFT JOIN
- recognize that FULL OUTER JOIN keeps rows from both sides
- understand that SELF JOIN is used for self-referencing tables
- understand that CROSS JOIN makes all possible combinations

Required checkpoint question

Why does a many-to-many relationship need a junction table?

Good beginner checkpoint answer

A many-to-many relationship needs a junction table because one row from the first table can connect to many rows in the second table, and one row from the second table can also connect to many rows in the first table. The junction table stores each connection clearly using foreign keys. For example, students and courses need a `course_enrollments` table because one student can join many courses, and one course can have many students.

28. Mini Assessment Before Sprint 14

Answer these questions before moving forward:

1. What is a primary key?
 2. What is a composite primary key?
 3. What is a foreign key?
 4. What is an orphaned record?
 5. Why can a table have multiple foreign keys?
 6. What is a one-to-one relationship?
 7. What is a one-to-many relationship?
 8. What is a many-to-one relationship?
 9. What is a many-to-many relationship?
 10. Why does many-to-many need a junction table?
 11. What is a self-referencing relationship?
 12. What does JOIN do?
 13. What does the `ON` condition do?
 14. What does INNER JOIN return?
 15. What does LEFT JOIN return?
 16. What does RIGHT JOIN return?
 17. What does FULL OUTER JOIN return?
 18. What is SELF JOIN used for?
 19. What does CROSS JOIN do?
 20. Why can CROSS JOIN become dangerous on large tables?
 21. What does `NULL` often mean in join results?
-

29. Sprint 13 Completion Checklist

Tick these when completed:

- ☐ I can explain what a primary key is.
- ☐ I can explain what a composite primary key is.

- ☐ I can explain what a foreign key is.
 - ☐ I understand that a table can have multiple foreign keys.
 - ☐ I understand what orphaned records are.
 - ☐ I can explain one-to-one relationships.
 - ☐ I can explain one-to-many relationships.
 - ☐ I can explain many-to-one relationships.
 - ☐ I can explain many-to-many relationships.
 - ☐ I know why a junction table is needed.
 - ☐ I understand self-referencing relationships.
 - ☐ I understand what JOIN means.
 - ☐ I can explain `ON` in a join.
 - ☐ I understand INNER JOIN.
 - ☐ I understand LEFT JOIN.
 - ☐ I understand RIGHT JOIN.
 - ☐ I understand FULL OUTER JOIN.
 - ☐ I understand SELF JOIN.
 - ☐ I understand CROSS JOIN.
 - ☐ I can create a simple linked-table example in PostgreSQL.
 - ☐ I am ready to move to Sprint 14.
-

30. Final Simple Meaning of Sprint 13

Sprint 13 teaches you how to connect tables using keys and relationships, and how to read connected data using joins. It is the bridge between simple single-table SQL and real relational database design.